
Software Requirements Specification

For

Eventalk – Event Organizer

Version 1.1 approved

Prepared by: Kelvin Chovatiya

Yash Kanzariya

Kris Kalariya

RK University

08-05-2026

Table of Contents

Table of Contents.....	i
Revision History.....	ii
1. Introduction	4
1.1 Purpose.....	4
1.2 Document Conventions	4
1.3 Intended Audience and Reading Suggestions	4
1.4 Project Scope	4
1.5 References.....	4
2. Overall Description.....	6
2.1 Product Perspective.....	6
2.2 Product Features	6
2.3 User Classes and Characteristics	6
2.4 Operating Environment.....	6
2.5 Design and Implementation Constraints.....	7
2.6 User Documentation	7
2.7 Assumptions and Dependencies	7
3. System Features.....	8
3.1 Authentication & User Management	8
3.1.1 Description and Priority	8
3.1.2 Stimulus/Response Sequences	8
3.1.3 Functional Requirements	8
3.2 Event Browsing & Catalog	8
3.2.1 Description and Priority	8
3.2.2 Stimulus/Response Sequences	8
3.2.3 Functional Requirements	9
3.3 Booking & Registration Engine	9
3.3.1 Description and Priority	9
3.3.2 Stimulus/Response Sequences	9
3.3.3 Functional Requirements	9
3.4 User Dashboard (My Tickets)	9
3.4.1 Description and Priority	9
3.4.2 Stimulus/Response Sequences	10
3.4.3 Functional Requirements	10
3.5 Admin Dashboard	10
3.5.1 Description and Priority	10
3.5.2 Stimulus/Response Sequences	10
3.5.3 Functional Requirements	10

4. External Interface Requirements	11
4.1 User Interfaces	11
4.2 Hardware Interfaces	11
4.3 Software Interfaces	11
4.4 Communications Interfaces.....	11
5. Other Nonfunctional Requirements	12
5.1 Performance Requirements	12
5.2 Safety Requirements	12
5.3 Security Requirements	12
5.4 Software Quality Attributes.....	12
6. Other Requirements	13
6.1 Database Requirements	13
6.2 Legal Requirements.....	13
Appendix A: Glossary	14
Appendix B: Analysis Models	15
B.1 Authentication Flow	15
B.2 Booking & Payment Flow	16
B.3 System Flowchart	17

Revision History

Name	Date	Reason For Changes	Version
Kelvin Chovatiya, yash Kanzariya, Kris Kalariya	Feb 20, 2026	Initial SRS Creation	v1.0
Kris Kalariya	Mar 6, 2026	Added authentication module (login/signup)	v1.1
Yash Kanzariya	Apr 1, 2026	Implemented event creation and event register feature	v1.2
Kelvin Chovatiya, Yash Kanzariya, Kris Kalariya	Apr 20, 2026	Updated UI, fixed major bugs, and converted project to ASP.NET MVC stack	v1.3

1.Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functional and non-functional requirements for **Eventalk**, a full-stack event management and ticketing web application built on the ASP.NET MVC framework. This document covers the scope of the ASP.NET MVC frontend (Razor Views), the C# controller/service layer, and the Microsoft SQL Server (MSSQL) database. It is intended to guide the development, testing, and maintenance of the version 1.1 release.

1.2 Document Conventions

- Requirements are uniquely identified with tags (e.g., REQ-AUTH-1, REQ-EVT-2).
- Priority is indicated as Critical, High, Medium, or Low.
- "Shall" indicates mandatory requirements; "should" indicates desirable but non-mandatory; "may" indicates optional.

1.3 Intended Audience and Reading Suggestions

- **Developers** – Focus on Sections 3 (System Features) and 4 (External Interfaces).
- **Project Managers / Stakeholders** – Read Sections 1, 2, and 5 first.
- **Testers** – Sections 3 and 5 (Non-functional).
- **Future maintainers** – Entire document + Appendix A (Glossary).

1.4 Project Scope

Eventalk is a complete web-based ticketing solution that enables end-users to securely authenticate, browse active events, book tickets, manage their personal registrations, and receive automated email confirmations. It also provides a protected administrative portal for event creation and management. The application is built on the ASP.NET MVC architecture with Entity Framework Core as the ORM and Microsoft SQL Server (MSSQL) as the database engine.

1.5 References

- ASP.NET Core MVC Documentation: <https://learn.microsoft.com/en-us/aspnet/core/mvc/>
- Entity Framework Core Documentation: <https://learn.microsoft.com/en-us/ef/core/>
- ASP.NET Core Identity: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity>
- SQL Server Documentation: <https://learn.microsoft.com/en-us/sql/sql-server/>
- MailKit / SMTP Documentation: <https://github.com/jstedfast/MailKit>

- Razor Pages Documentation: <https://learn.microsoft.com/en-us/aspnet/core/razor-pages/>

2.Overall Description

2.1 Product Perspective

Eventalk is a new, self-contained product. It operates as a client-server system:

- **Client:** Razor Views (.cshtml) rendered server-side by ASP.NET Core MVC.
- **Server:** C# Controllers and Service classes handling business logic.
- **Database:** Microsoft SQL Server (MSSQL) for persistent relational data storage, accessed via Entity Framework Core.

2.2 Product Features

The major features of Eventalk are divided into User and Admin functionalities:

User Features:

- Secure Authentication (Sign up, Login, Encrypted Passwords).
- Password Recovery Pipeline (SMTP email token generation).
- Event Catalog (Browse events with dynamic category-based image mapping).
- Booking Engine (Real-time seat deduction and duplicate prevention).
- User Dashboard (View purchased tickets and event details).

Admin Features:

- Admin Dashboard (Create, Edit, and Delete event listings).
- Registration Overview (View all user registrations across events).

2.3 User Classes and Characteristics

- **End Users (Attendees):** Users who browse events and book tickets. They require an intuitive UI and instant ticketing feedback. Technical expertise: Low to Medium.
- **Administrators:** Internal users responsible for managing the event catalog. They require secure, elevated access privileges to execute CRUD operations on the database. Technical expertise: Medium to High.

2.4 Operating Environment

- **Client OS:** Modern web browsers (Chrome, Firefox, Safari, Edge).
- **Server Runtime:** .NET 8 / ASP.NET Core runtime hosted on Windows Server (IIS) or Linux (Kestrel).
- **Database:** Microsoft SQL Server 2019+ or SQL Server Express.

- **Network:** Internet connection required for API calls and email delivery.

2.5 Design and Implementation Constraints

- Frontend must use ASP.NET Core MVC Razor Views with Bootstrap 5 for responsive layout.
- Backend must use ASP.NET Core Identity for authentication and role-based authorization.
- Emails must be processed via MailKit/SMTP using the built-in IEmailSender abstraction.
- All passwords must be hashed using ASP.NET Core Identity's PasswordHasher (PBKDF2) prior to database insertion.

2.6 User Documentation

- In-browser onboarding / navigation flow.
- README.md with setup and troubleshooting guide.
- API Documentation section in repository.

2.7 Assumptions and Dependencies

- A valid SMTP server/account is available for MailKit email delivery.
- A Microsoft SQL Server instance is reachable from the application server.
- Internet connectivity is required for authentication and email confirmation.

3.System Features

3.1 Authentication & User Management

3.1.1 Description and Priority

- Allows users to register, log in, and securely recover their accounts.
- **Priority: High.**

3.1.2 Stimulus/Response Sequences

- User submits registration form → AccountController validates model → ASP.NET Identity creates user with hashed password (PBKDF2) → Saves to MSSQL via EF Core → System sends Welcome Email via MailKit/SMTP.
- User requests password reset → System generates secure token → System sends reset link via email.

3.1.3 Functional Requirements

Req. ID	Description	Priority
REQ-AUTH-1	The system shall allow users to register using Name, Email, and Password.	High
REQ-AUTH-2	The system shall hash all passwords using ASP.NET Core Identity's PBKDF2 PasswordHasher prior to database insertion.	High
REQ-AUTH-3	The system shall authenticate users via ASP.NET Core Cookie Authentication, issuing a secure encrypted session cookie upon successful login.	High
REQ-AUTH-4	The system shall allow users to reset their password via an SMTP-delivered secure URL token.	High

3.2 Event Browsing & Catalog

3.2.1 Description and Priority

- The core interface for users to view available events.
- **Priority: High.**

3.2.2 Stimulus/Response Sequences

- User navigates to Event Catalog → System fetches active events → UI displays event cards mapped with category-specific high-resolution imagery.

3.2.3 Functional Requirements

Req. ID	Description	Priority
REQ-EVT-1	The system shall display a list of events with Title, Date, Location, and Available Seats.	High
REQ-EVT-2	The system shall dynamically map high-quality category images to events lacking specific database URLs.	Medium

3.3 Booking & Registration Engine

3.3.1 Description and Priority

- Manages the selection and finalization of event tickets.
- **Priority: Critical.**

3.3.2 Stimulus/Response Sequences

- User clicks "Register" → System verifies seat availability → System creates Registration document → System dispatches digital ticket via email.

3.3.3 Functional Requirements

Req. ID	Description	Priority
REQ-BKG-1	The system shall prevent registration if availableSeats is strictly 0 or less.	Critical
REQ-BKG-2	The system shall query the database to prevent a user from registering for the exact same event multiple times.	Critical
REQ-BKG-3	Upon successful database insertion, the system shall asynchronously dispatch an email confirmation containing event details.	High

3.4 User Dashboard (My Tickets)

3.4.1 Description and Priority

- A protected area for users to view their transaction history and upcoming events.

- **Priority: High.**

3.4.2 Stimulus/Response Sequences

- Authenticated user navigates to dashboard → System queries Registration collection by User ID
→ System populates associated Event data → UI renders ticket list.

3.4.3 Functional Requirements

Req. ID	Description	Priority
REQ-DASH-1	The system shall display all Registration records associated with the currently logged-in user's Identity ClaimsPrincipal, enforced via [Authorize] attribute.	High
REQ-DASH-2	The system shall populate Event details (Title, Date, Location) within the Registration payload to display complete digital tickets.	High

3.5 Admin Dashboard

3.5.1 Description and Priority

- Administrative control for content and user management.
- **Priority: Critical.**

3.5.2 Stimulus/Response Sequences

- Admin authenticates → Navigates to Admin portal → Submits new Event form → AdminController validates ModelState → EF Core inserts new Event record into MSSQL.

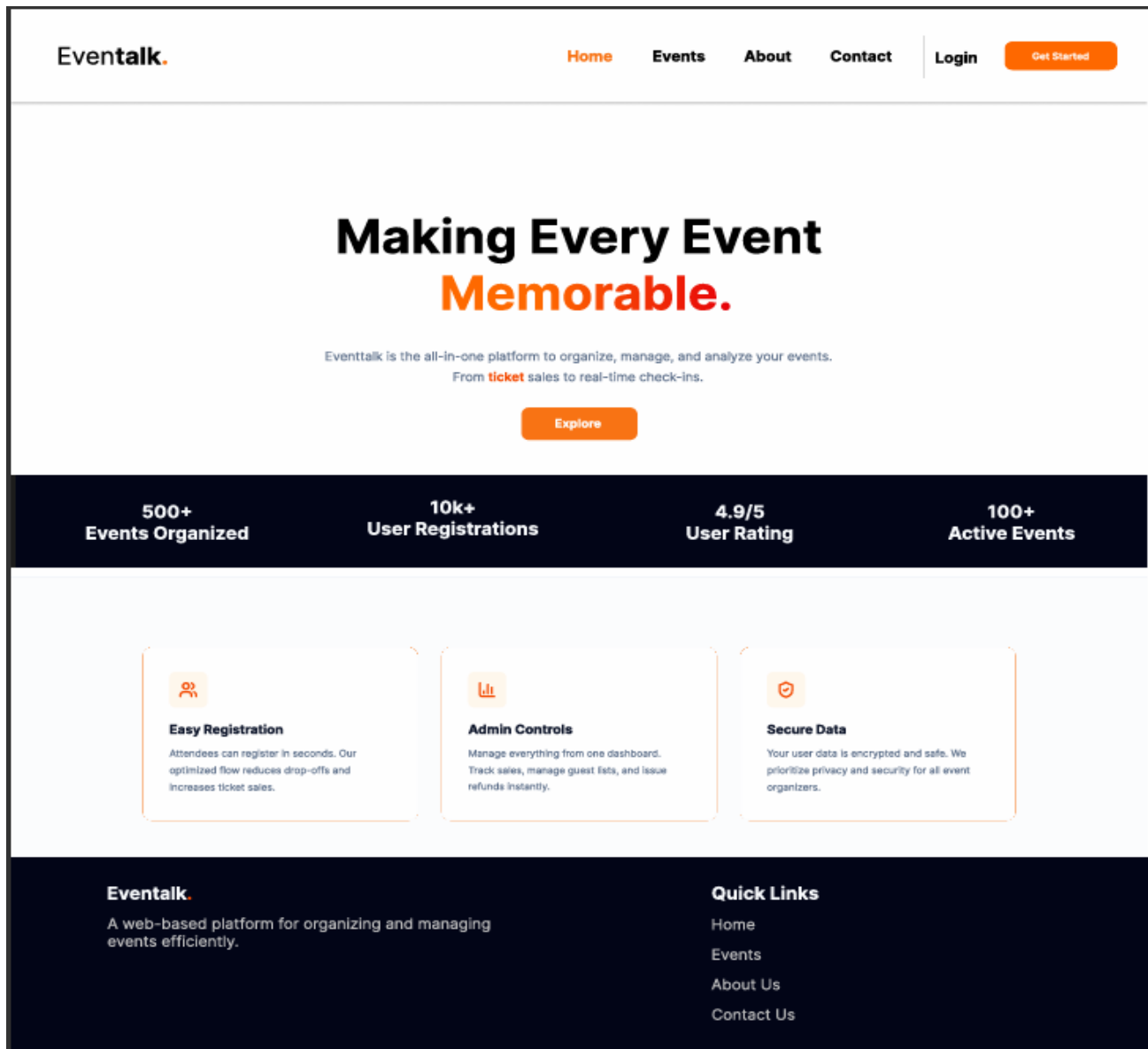
3.5.3 Functional Requirements

Req. ID	Description	Priority
REQ-ADM-1	The system shall restrict access to Admin routes strictly to users with the role of admin.	Critical
REQ-ADM-2	Administrators shall be able to Add, Edit, and Delete event listings in the database.	Critical
REQ-ADM-3	Administrators shall be able to view an aggregated list of all user registrations across all events.	High

4.External Interface Requirements

4.1 User Interfaces

- **Frontend Framework:** ASP.NET Core MVC Razor Views (.cshtml) with strongly-typed ViewModels.
- **Routing:** ASP.NET Core MVC attribute routing and conventional routing.
- **Styling:** Bootstrap 5 for responsive layout structuring, served as static files via wwwroot.
- **Icons:** TempData / ModelState for success/error messages rendered in partial views.
- **Feedback:** Toast notifications for success/error states.



4.2 Hardware Interfaces

- **Network Radio:** Required for HTTP requests to the backend API.

4.3 Software Interfaces

- **Database:** Microsoft SQL Server accessed via Entity Framework Core (Code-First, DbContext).
- **Email Communication:** MailKit configured with an SMTP service via IEmailSender abstraction.

4.4 Communications Interfaces

- **Protocol:** HTTP/1.1.
- **Data Format:** JSON for all API request bodies and responses.
- **Security:** HTTPS required for production; standard HTTP allowed for local development.
- **Email:** SMTP via MailKit for transactional email delivery.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **API Latency:** API responses should generally occur within 500ms under normal load conditions.
- **Concurrent Users:** The system should support at least 100 simultaneous users without performance degradation.

5.2 Safety Requirements

- **Asynchronous Failsafes:** If the external SMTP email server fails, primary database transactions must still complete successfully without terminating the request process.
- **Stock Safety:** The system must prevent the booking of events where availableSeats is 0 or less.

5.3 Security Requirements

- **Encryption:** Passwords must be hashed using ASP.NET Core Identity's PBKDF2 PasswordHasher before storage.
- **Token Management:** Access must be controlled via encrypted ASP.NET Core Cookie Authentication (Data Protection API).
- **Secrets Management:** Connection strings, SMTP credentials, and Data Protection keys must be isolated in appsettings.json (or User Secrets) and excluded from version control.
- **CORS:** Cross-Origin Resource Sharing must be configured to allow requests only from authorized clients.

5.4 Software Quality Attributes

- **Maintainability:** Codebase is modularized with separate route, controller, and model layers to facilitate future updates.
- **Portability:** ASP.NET Core MVC application can be deployed to Windows (IIS), Linux (Kestrel/Nginx), or containerized via Docker.
- **Reliability:** The application is stateless per-request and can be restarted without data loss due to persistent MSSQL storage and EF Core migrations.

6. Other Requirements

6.1 Database Requirements

- **Schema:** Strict Entity Framework Core Code-First models must be defined for ApplicationUser, Event, and Registration, with migration scripts versioning all schema changes.
- **Relationships:** Registration documents must maintain referential integrity by dynamically referencing valid User IDs and Event IDs.

6.2 Legal Requirements

- **Privacy:** User passwords and personal details must not be logged in plain text.
- **Data Retention:** User registration data must be retained only as long as necessary for service delivery.

Appendix A: Glossary

- **JWT:** JSON Web Token; previously used in the React version for session management. In the ASP.NET version, sessions are managed via Cookie Authentication.
- **ASP.NET MVC Stack:** ASP.NET Core MVC, Entity Framework Core, Razor Views, MSSQL – the full-stack architecture used by Eventalk (ASP.NET edition).
- **Mongoose:** An ORM (Object-Relational Mapper) for .NET; in this project replaced by Entity Framework Core for MSSQL access.
- **MailKit:** A cross-platform .NET library for sending emails via SMTP, used in place of Nodemailer.
- **PBKDF2 / PasswordHasher:** The password-hashing algorithm used by ASP.NET Core Identity to securely store user passwords (replaces bcrypt from the React version).
- **REST:** Representational State Transfer; the architectural style used for the backend API.
- **SMTP:** Simple Mail Transfer Protocol; used for sending transactional emails.
- **ASP.NET Core Routing:** ASP.NET Core MVC conventional and attribute routing that enables server-side navigation.
- **Vite / .NET SDK:** A fast frontend build tool used in the React version; replaced by the .NET SDK build pipeline in the ASP.NET edition.
- **CRUD:** Create, Read, Update, Delete – the four basic database operations.

Appendix B: Analysis Models

B.1 Authentication Flow

The following sequence diagram illustrates the user registration and OTP verification flow between the Client, API Gateway, Email Service, and Database.

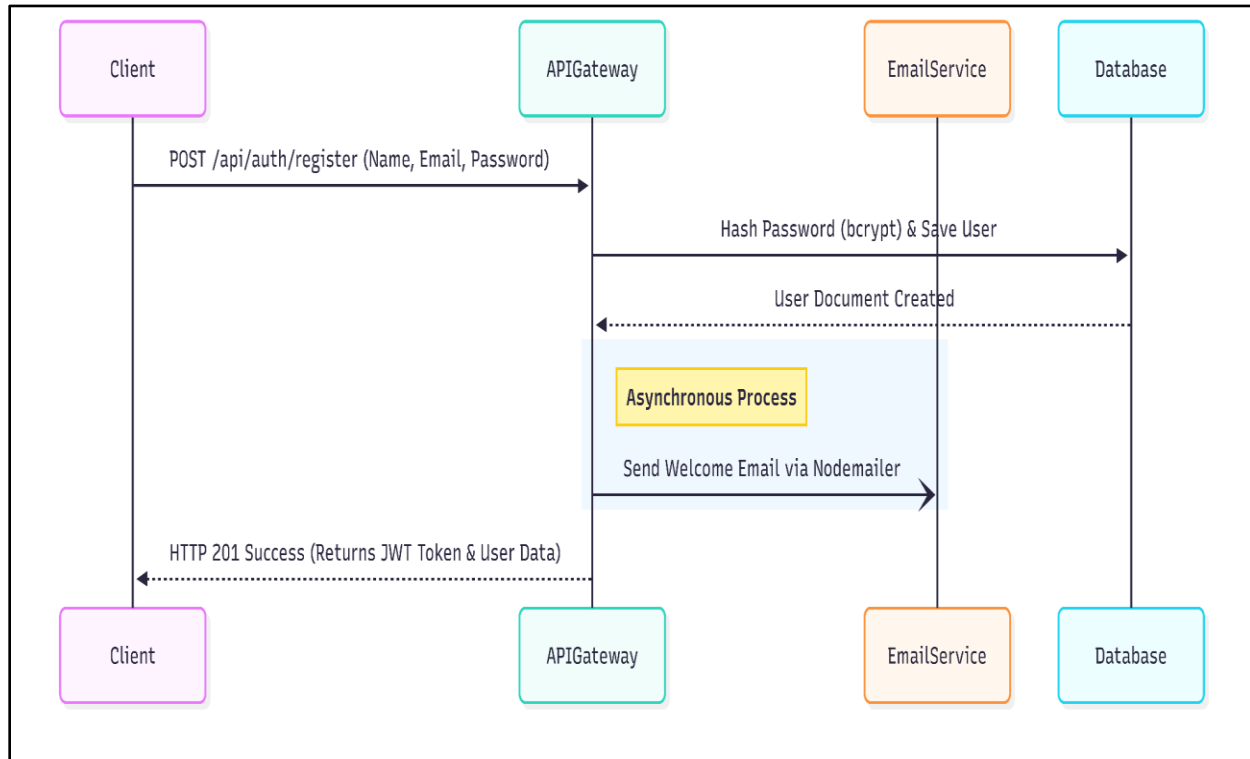


Figure 1: Authentication & Registration Sequence Diagram

B.2 Booking & Payment Flow

The following sequence diagram illustrates the end-to-end ticket booking and email confirmation flow between the Client, API Gateway, Email Service, and Database.

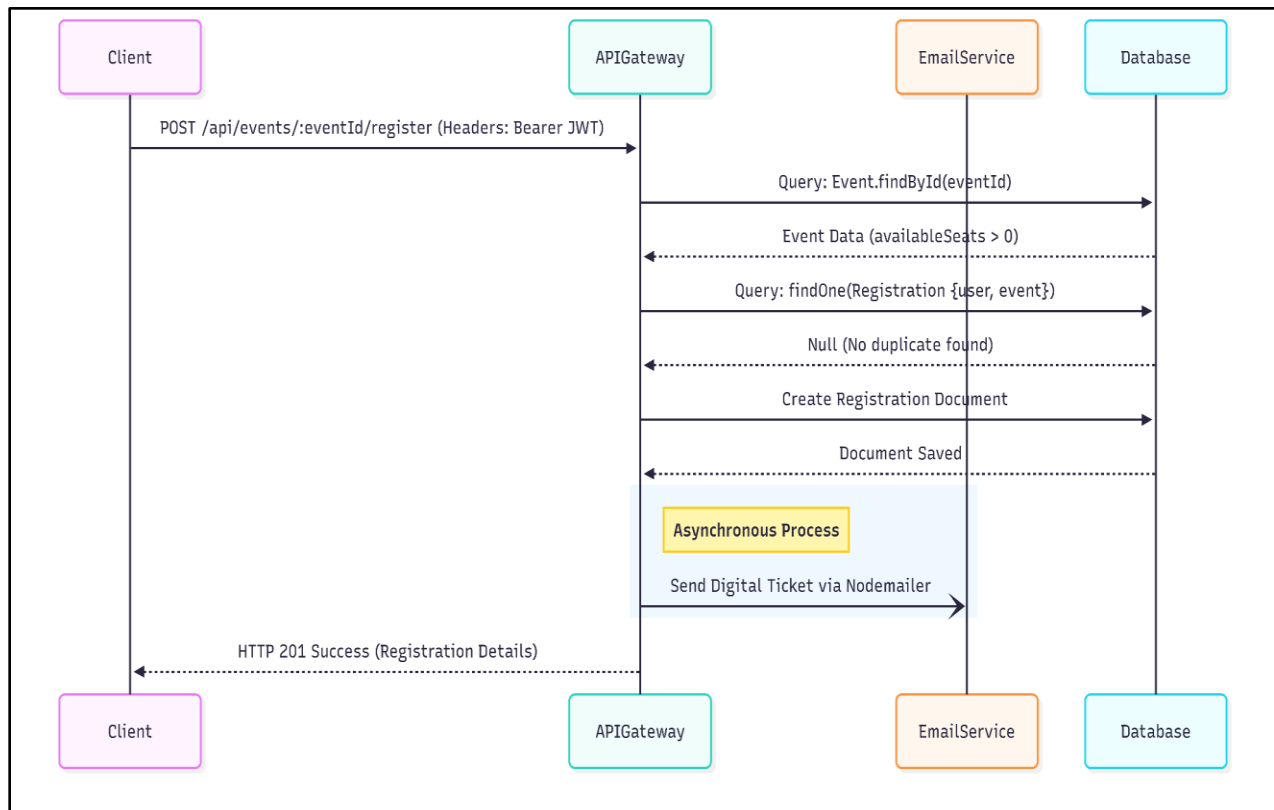


Figure 2: Booking & Email Confirmation Sequence Diagram

B.3 System Flowchart

The following flowchart illustrates the overall system flow for both End Users (Attendees) and Administrators, from login through to their respective operations.

Figure 3: Overall System Flowchart – User & Admin Paths

